Predicting Task Difficulty Without Rollouts

Stefan Krsteski & Charlotte Meyer
Andromede AI, EPFL
{stefan, charlotte}@andromede.ai {stefan.krsteski, charlotte.meyer}@epfl.ch

arXiv:2608.05797v3 [cs.LG] 15 Oct 2026

Abstract

A fundamental challenge in evaluating and training autonomous agents is measuring the intrinsic difficulty of the tasks they attempt. Estimating this quantity can be useful for environment designers creating synthetic data or benchmarks, as well as for constructing training curricula, thereby offering a way to reduce compute costs. Such an estimation becomes increasingly important as agents (particularly LLM-based) move into longer-horizon domains, where empirical trial-and-error becomes a severe computational bottleneck. However existing work is largely confined to static tasks and relies on evaluation metrics that, as we show, can give an incomplete picture of predictive performance. In this paper we study *ex ante* difficulty prediction across 17 agentic benchmarks spanning coding, mathematics, machine learning, web navigation, function calling, and other domains. We find that AUC can mask poor difficulty estimates, identify token-level entropy as a useful predictive signal, and demonstrate how residuals between expected and observed difficulty can expose environment flaws such as contamination and infeasibility.

1 Introduction

As LLM agents rapidly improve, the performance frontier is pushed toward domains that are increasingly hard to design, validate, and simulate. This shift creates a financial and logistical bottleneck for environment designers. Evaluating frontier systems on complex, long-horizon tasks can require hours of simulated interaction and many model generations per attempt (Kwa et al., 2026). Consequently, the field faces a paradox: progress requires carefully calibrated environments, but verifying their difficulty empirically is increasingly prohibitive. This paper argues for a shift toward *ex ante* difficulty prediction, with the goal of estimating whether a task is likely to be hard before spending large amounts of compute on evaluation or training.

Recent work has used a model's uncertainty to detect data contamination in text-based benchmarks like question-answering (Li, 2023; Xu et al., 2024; Dong et al., 2024). Yet contamination can be viewed as an instance of a more fundamental variable, namely task difficulty. If a model has effectively memorized an item, the task becomes easier for reasons unrelated to the intended capability. In interactive environments, difficulty can affect both success rates and the strategies agents use. In human settings, high task complexity can push people to alter their strategies, such as resorting to dishonesty (Mazar et al., 2008). Similarly, when language-model agents operate near the edge of their competence, they may exploit shortcuts (Skalse et al., 2022; Helff et al., 2026). Additionally, difficulty can reveal when an evaluation task is misaligned with its intended purpose. A task may register as difficult because it exceeds an agent's capabilities, or simply because the environment precludes a successful solution. Conversely, a task may appear easy because it is familiar or prone to shortcuts. We therefore posit that difficulty serves as a common variable connecting task design and agent behavior.

If difficulty could be estimated before rollouts, environment designers could construct calibrated evaluations without first spending the cost of full empirical validation. Training pipelines could also build curricula of progressively harder tasks (Bengio et al., 2009), while evaluations could better contextualize success and failure (Kwa et al., 2026).

Despite this motivation, pre-rollout difficulty prediction remains underdeveloped. To the best of our knowledge, only one prior paper directly studies this setting (Ge et al., 2026), specifically in coding benchmarks. This leaves two open questions: how should such estimates be evaluated, and do they transfer across benchmark families? We show that response AUC can be misleading, while task-level rank metrics give a more cautious picture. In this work, we broaden the setting beyond coding tasks to seven additional benchmark types and take a diagnostic view of difficulty prediction. Our core contributions are:

- **Evaluation of difficulty prediction**, which shows that response AUC can remain high even when task difficulties are poorly estimated. Under rank correlation, entropy is the strongest single feature family, while the combined feature set performs best overall, reaching Spearman $\rho = 0.399$ under K-fold evaluation ($p < 0.001$ vs. baseline) and $\rho = 0.225$ out of distribution (§4, §5).
- **Residual diagnostics for hidden task factors**, showing how gaps between predicted and observed difficulty can guide follow-up analysis. Positive residuals point to unexpectedly hard tasks, such as feasibility or evaluator failures, while negative residuals point to unexpectedly easy tasks, such as familiarity (§5.3).
- **An open multi-benchmark corpus** with 415,470 agent-task outcomes across 17 benchmarks, including task metadata, verifier summaries, textual descriptions, and IRT-estimated difficulties for reuse beyond this paper in behavioral analysis, environment design, and training (§3.1).

2 Related work

Taking the rollout as the point of reference, we distinguish three sets of methods by when the difficulty signal is observed: after, during, and before execution.

After rollout. Agentic benchmarks often use empirical success rates, rewards, or other trajectory-level quantities such as number of actions, steps, and elapsed time to estimate difficulty, with human completion time serving as a gold standard (Kwa et al., 2026; Liu et al., 2026; Ho et al., 2025). These methods make difficulty explicit, but their signal is observed only after agents or humans interact with the task. Our work instead studies whether difficulty can be predicted from information available before running agents.

Adaptive methods. Curriculum learning relies on difficulty to order training examples or environments for greater sample efficiency (Bengio et al., 2009; Vygotsky, 1978; Portelas et al., 2020). Many online variants define difficulty relative to the current learner. Some methods, such as self-paced and automated curriculum learning, use loss, prediction error, or temporal-difference (TD) error as the difficulty signal (Kumar et al., 2010; Graves et al., 2017; Jiang et al., 2021). Others, such as teacher-student curricula and unsupervised environment design, use reward, regret or success to select tasks that are neither too easy nor too hard (Florensa et al., 2018; Dennis et al., 2020; Parker-Holder et al., 2022; Beukman et al., 2024; Tio et al., 2023; Mahrooghi et al., 2026; Matiisen et al., 2019; Bae et al., 2026). Nonetheless, these methods require executing tasks, often inside an additional teacher-student loop, which can further compound the rollout overhead we aim to circumvent.

Before rollout. More distant work on static benchmarks motivates using uncertainty signals from language models, including perplexity and entropy (Gonen et al., 2023; Li, 2023; Xu et al., 2024; Dong et al., 2024; Spiesberger et al., 2026; Zhu et al., 2026; Ostmeier et al., 2026; Català et al., 2026; Feng et al., 2026; Truong et al., 2025). These settings are not interactive and do not cover agentic benchmarks. To the best of our knowledge, Agent Psychometrics is the only prior work that directly studies *ex ante* difficulty prediction for agentic tasks, focusing on coding benchmarks (Ge et al., 2026). We test broader generalization by evaluating predictors across seven additional benchmark families. We also evaluate a broader feature set, including token-level entropy, cross-scorer disagreement, structural features, and embeddings.

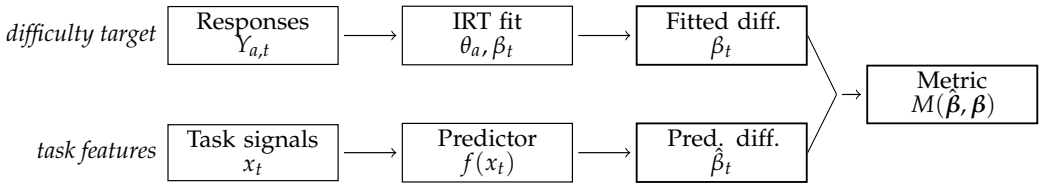


Figure 1: **Overview of the difficulty prediction pipeline.** Upper branch: binary agent success and failure outcomes are used to fit IRT and estimate each task's difficulty, β_t. Lower branch: task features are used to predict difficulty, $\hat{\beta}_t$, and evaluation compares the predicted and IRT-estimated difficulty vectors.

3 Methods

We decompose the prediction pipeline into two steps: constructing a difficulty target from observed agent-task outcomes, then training a predictor to estimate this target from task descriptions and other pre-rollout features (Figure 1).

3.1 Dataset

Our dataset (Krsteski et al., 2026)[1] consists of agent-task outcomes from 17 agentic benchmarks, listed in Appendix A.1. Each instance records one trial of one agent configuration on one task, labeled by success under the benchmark's scoring rules. We use *agent* to mean a model (LLM) together with the scaffold through which it acts.

Statistic	Value
# Benchmarks	17
# Benchmark types	8
# Tasks	5,230
# Models	216
# Scaffolds	90
# Agents	497
Task dates	2010–2026
Agent dates	2023–2026
Avg. trials / agent-task	1.2
Mean success rate	0.539

Table 1: Summary of the dataset.

Table 1 summarizes the main analysis subset used throughout the main-text results. It contains 5,230 tasks from 17 benchmarks, evaluated by 497 agent configurations built from 216 models and 90 scaffolds. For each task, the task, environment, and action-space fields are stored as string descriptions derived from the benchmark, either directly or via an LLM when missing. These benchmarks span coding and software engineering, function calling, web navigation, computer use, terminal tasks, math, ML, cybersecurity, and general-purpose task suites. Task dates denote the task release or source date when available and range from 2010 to 2026. Agent dates denote the model-weight release date when available, otherwise the first public availability date, and range from 2023 to 2026. Each trial is treated as an independent Bernoulli observation under the benchmark scoring rules, with 1.2 trials per agent-task pair on average.

3.2 Difficulty target

IRT is a psychometric framework that models the relationship between an individual's latent trait (e.g., knowledge, ability, anxiety) and their responses to specific test items (Embretson & Reise, 2025). In our case, the "individual" is the agent and the "test items" are the tasks. Let $\mathcal{A} = a_1, \ldots, a_N$ denote the set of (N) agent configurations and $\mathcal{T} = t_1, \ldots, t_M$ the set of (M) tasks. The data form a partially observed binary response matrix

$$Y_{a,t} = \begin{array}{c} \\ a_1 \\ a_2 \\ \vdots \\ a_N \end{array} \begin{array}{c} \begin{array}{cccccc} t_1 & t_2 & t_3 & \cdots & t_M \end{array} \\ \begin{bmatrix} 1 & 0 & \mathrm{x} & \cdots & \mathrm{x} \\ 0 & 1 & 0 & \cdots & \mathrm{x} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \mathrm{x} & \mathrm{x} & 1 & \cdots & 1 \end{bmatrix} \end{array}$$

[1]This work uses a preliminary version of the corpus.

where 1 denotes success, 0 denotes failure, x denotes a missing evaluation, and Ω is the set of observed agent-task pairs. Each row is a unique agent configuration, such as a base model with a scaffold, and each column is a unique task. The goal is to model the probability of success by learning one scalar θ_a for each agent and one scalar β_t for each task. In the one-parameter logistic IRT model (1PL; Rasch model (Rasch, 1960)), this is written as

$$P(Y_{a,t} = 1 \mid \theta_a, \beta_t) = \sigma(\theta_a - \beta_t), \tag{1}$$

where θ_a represents agent capability, β_t represents task difficulty and $\sigma(z) = 1/(1 + \exp(-z))$ is the sigmoid function. This form gives the desired inductive bias because success depends on the relative position of capability and difficulty on a shared scale. If $\theta_a = \beta_t$, capability and difficulty are balanced; if $\theta_a > \beta_t$, capability exceeds difficulty; and if $\theta_a < \beta_t$, difficulty exceeds capability. The resulting β_t is therefore a scalar summary of apparent task difficulty as inferred from observed outcomes, and may absorb task-side covariates such as feasibility, strictness, familiarity, and noise, just as θ_a may absorb agent-side covariates such as scaffold effectiveness, tool affordances, and noise. With an interpretable predictor and an appropriate setup, this lets us ask which features account for different parts of the fitted difficulty.

The model is trained on observed pairs $(a, t) \in \Omega$ by optimizing the evidence lower bound (ELBO) via stochastic variational inference (SVI) (Hoffman et al., 2013) and the reparameterization trick (Kingma & Welling, 2013). We place zero-centered Gaussian priors over all parameters and approximate the posterior distribution over these parameters with independent Gaussian variational distributions:

$$q(\theta_a) = \mathcal{N}(\mu_{\theta_a}, \sigma^2_{\theta_a}), \qquad q(\beta_t) = \mathcal{N}(\mu_{\beta_t}, \sigma^2_{\beta_t}).$$

Because the model depends strictly on the difference $\theta_a - \beta_t$, the parameters can shift arbitrarily without changing the probabilities. To resolve this, we anchor the scale by centering the abilities θ_a at zero. We then use the resulting task difficulty estimates (β) as the target for downstream prediction. The fitted scores achieve a response AUC of 0.937 on observed pairs, indicating that the 1PL model captures much of the observed response structure. Finally, as difficulties vary across different benchmarks, we compute a standardized version of these values within each benchmark to obtain $z(\beta_t)$. More details are provided in Appendix A.2.

3.3 Task features

The target is the IRT-estimated difficulty β_t, which we standardize within-benchmark to $z(\beta_t)$ for our main experiments, as described above. The practical constraint is that every feature must be less expensive to compute than running rollouts on the task. Let C_t denote the full task-side context for task t, consisting of the environment description, action-space description, and task description. Feature families use C_t either directly or through a reasoning trace R_t generated from C_t by Claude Sonnet 4.6 with a 4k-token cap.

We use five feature sets throughout the experiments. The *random* control samples $x_t \sim \mathcal{N}(0, 1)$ independently from the task. The *baseline* setting uses only the context length, $x_t^{\text{len}} = \log(1 + |C_t|)$, to control for the amount of task text. The *embedding* family maps the reasoning trace to $e_t = \phi(R_t)$, where ϕ is `bge-base-en-v1.5`. We reduce this representation with fold-aware PCA to avoid overfitting.

The *entropy* setting uses an open-weight scorer model. Given $R_t = (r_1, \ldots, r_L)$, the scorer defines $p_\ell(v) = p(v \mid C_t, r_{<\ell})$ at each position ℓ, where v is a token from the model's vocabulary. We compute Shannon entropy (Shannon, 1948) as

$$H_\ell = -\sum_v p_\ell(v) \log p_\ell(v),$$

using the renormalized top-100 next-token distribution for faster computation. The primary entropy configuration aggregates $(H_1, \ldots, H_L)$ from Qwen3-8B-Base over the reasoning trace R_t. The *full* setting concatenates entropy summaries from a five-model scorer panel,

cross-scorer disagreement features, benchmark metadata, structural properties of C_t, and the embedding features defined above. Appendix A.3 gives the scorer panel.

To see that these features satisfy the cost constraint from above, consider a 100-task benchmark. Establishing empirical difficulty at a density of 10 outcomes per task requires 1,000 full executions. Assuming a representative 100,000-token context per execution (Bai et al., 2026), this process demands approximately 1.0×10^8 input tokens. In contrast, estimating difficulty *ex ante* using the features proposed here (accounting for the reasoning trace) consumes roughly 4.0×10^6 tokens for the same 100 tasks. Under these assumptions, the feature pass uses $25\times$ fewer tokens than empirical difficulty estimation, and the efficiency grows with the number of outcomes required per task.

4 Evaluation setup

4.1 Prediction metrics

The preceding sections define a target task difficulty β_t, obtained from observed agent-task outcomes, and a predicted difficulty $\hat{\beta}_t = f(x_t)$, obtained from task features available before running rollouts. The evaluation asks how well $\hat{\beta}_t$ recovers β_t.

We report Spearman's ρ between predicted and IRT-estimated task difficulty. We use ρ_β for raw fitted difficulty and $\rho_{z(\beta)}$ for within-benchmark standardized difficulty. We also report within-benchmark pairwise accuracy (pAcc) as a complementary ordering metric. Let q_t denote the target being evaluated, either β_t or $z(\beta_t)$, and let $\hat{q}_t$ denote its prediction. Then pAcc is the fraction of within-benchmark task pairs for which the predicted ordering matches the target ordering:

$$\text{pAcc} = \frac{1}{|\mathcal{P}|} \sum_{(i,j)\in\mathcal{P}} \mathbb{1}\left[(\hat{q}_i - \hat{q}_j)(q_i - q_j) > 0\right], \tag{2}$$

where $\mathcal{P}$ is the set of task pairs within the same benchmark. K-fold evaluation (KF) measures in-distribution performance, where tasks from the target distribution are available during training. Leave-one-benchmark-out (LOBO) measures out-of-distribution performance, where the target benchmark is unseen during training.

4.2 AUC can be misleading

Prior work (Ge et al., 2026) evaluates difficulty predictors through AUC. Using its probabilistic interpretation, AUC measures the probability that a randomly chosen successful attempt receives a higher score than a randomly chosen unsuccessful attempt. For a successful attempt by agent a on task t, and an unsuccessful attempt by agent a' on task t', the IRT score gives

$$\begin{aligned} \text{AUC} &= P(\theta_a - \hat{\beta}_t > \theta_{a'} - \hat{\beta}_{t'}) \\ &= P(\theta_a - \theta_{a'} > \hat{\beta}_t - \hat{\beta}_{t'}). \end{aligned} \tag{3}$$

This expression shows that AUC here is affected by both the difference in agent abilities and the difference in predicted task difficulties. Consider a degenerate case where the predictor simply assigns the same prediction to all tasks. This makes $\hat{\beta}_t = \hat{\beta}_{t'}$ for every pair of sampled attempts, meaning

$$\text{AUC} = P(\theta_a > \theta_{a'}). \tag{4}$$

Thus, AUC can remain high even when the predictor contains no task-specific difficulty information. We demonstrate this failure mode empirically on the observed response matrix by comparing fixed difficulty assignments against the fitted IRT target and observed responses, shown in Table 2. The oracle row uses the IRT-estimated β, establishing an upper bound of 0.937 AUC for this response matrix. Predicting a single constant difficulty for every task still yields an AUC of 0.715, confirming the constant difficulty case in Equation 4. This

Estimator	AUC ↑	ρ_β ↑	$\rho_{z(\beta)}$ ↑	pAcc ↑
Oracle	0.937	1.000	1.000	1.000
Bench. avg.	0.806	0.519	–	0.000
Constant	0.715	–	–	0.000
Shuffled	0.610	-0.011	-0.005	0.500

Table 2: **Response AUC remains high for poor difficulty estimates.** Benchmark avg. assigns each task its benchmark-average. ρ_β and $\rho_{z(\beta)}$ compare against raw and within-benchmark-standardized β. pAcc is within-benchmark pairwise accuracy. Dashes indicate undefined. Note that this is illustrative, all later predictions target $z(\beta)$ instead of β.

inflation is further exaggerated by the benchmark-average estimator. Simply assigning every task the mean difficulty of its parent benchmark achieves an AUC of 0.806 and a non-trivial rank correlation (ρ_β of 0.519). This estimator cannot differentiate tasks *within* the benchmark, giving a strict pAcc of 0.000. Because the practical goal is to help benchmark builders rank and select candidate tasks within their specific evaluation settings, recovering this signal is essential. To remove this confound and evaluate task-level difficulty more directly, all subsequent training and prediction target within-benchmark standardized difficulty, $z(\beta)$.

5 Results

5.1 Difficulty is predictable from task features

Figure 2 reports rank recovery across the 17-benchmark corpus using Spearman ρ and ridge regression (Appendix A.4 compares alternative estimators). KF measures in-distribution performance, where similar tasks are available during training. LOBO measures out-of-distribution performance on an unseen target benchmark. The prediction target is $z(\beta)$.

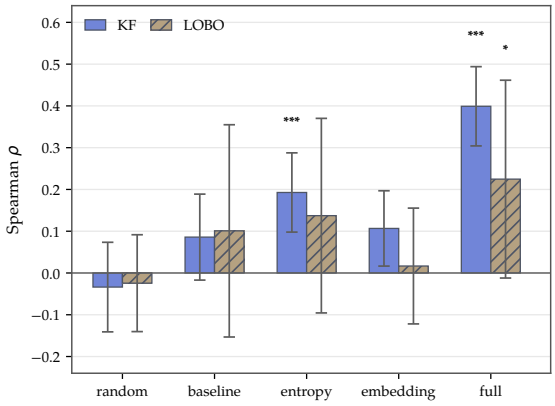
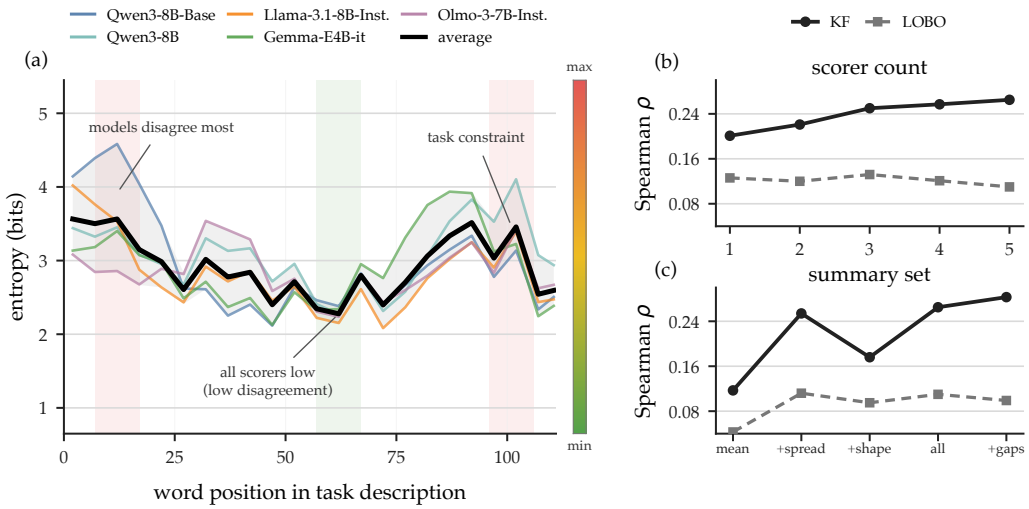


Figure 2: **Difficulty prediction performance.** The target is $z(\beta)$. Spearman ρ is shown under 50-fold KF and LOBO evaluation. Error bars show split standard deviations. With 50 folds, each test fold contains roughly 100 tasks, leading to substantial uncertainty. Stars mark one-sided paired Wilcoxon tests against the baseline at significance levels $^*\alpha = 0.05$, $^{**}\alpha = 0.01$, and $^{***}\alpha = 0.001$.

The random control stays near zero, while the length baseline captures only weak signal with $\rho = 0.086$ under KF and $\rho = 0.101$ under LOBO. The entropy features improve over this baseline, reaching $\rho = 0.193$ under KF and $\rho = 0.137$ under LOBO. Embedding features are weaker in this setup, with $\rho = 0.107$ under KF and $\rho = 0.017$ under LOBO, close to the length baseline. This suggests that these semantic representations capture little additional difficulty signal here, although stronger embedding models may change this conclusion. Combining feature families gives the strongest KF result, $\rho = 0.399$. Under LOBO, the same full model reaches $\rho = 0.225$. Thus, task features encode real difficulty signal, but transfer to unseen benchmarks remains limited. Taken together, the results point to a practical near-term setting in which an evaluated task suite already exists (with executed rollouts) and new candidate tasks can be ranked with moderate confidence.



Task (a). Consider an axis-aligned 4 dimensional cube with side length 4, subdivided into 4^4 unit cubes. Cubey starts at a random unit cube. A fish is independently placed in a uniformly random unit cube, possibly the same as Cubey's starting position. Compute the expected time for Cubey to catch the fish, assuming he moves optimally.

Figure 3: **Entropy profiles and feature ablations. (a)** An example task from MathArena, showing the entropy profile over its span. Colored lines are different models, black is their average, and the textual excerpt is given in the blue box. **(b)** Scorer-count ablation using one fixed order. **(c)** Shows an ablation of feature derivation from the entropy profile.

5.2 Entropy ablations

As illustrated in Figure 3**(a)**, entropy can be considered a trajectory rather than a static metric. Following the principle that conditioning reduces entropy (Cover, 1999), additional context should reduce expected next-token uncertainty. Consequently, global statistics such as the mean and variance may miss local structure in the entropy profile. For instance, a highly unexpected, difficulty-inducing condition that appears late in the context may have a lower entropy magnitude than the initial tokens. Thus, any aggregate (compression) is susceptible to losing this information.

The critical information may therefore lie in local discrepancies and structural anomalies. Figure 3**(c)** supports this intuition. Beyond the mean, *spread* adds distributional summaries such as standard deviation, quantiles, and top-k means, *shape* adds trajectory summaries such as slope, first- and last-decile means, and total variation, and *gaps* adds pairwise absolute differences between scorer summaries. These trajectory summaries improve in-distribution performance over mean-only entropy while preserving or improving LOBO performance. This suggests that the shape of the entropy trajectory carries useful information. Future work could model entropy explicitly as a time series, for example with frequency-domain or learned sequence features. With sufficient data, deep learning architectures such as 1D-CNNs or RNNs could also learn such features directly.

At the same time, ensembling models introduces a counterintuitive dynamic. Figure 3**(b)** shows that increasing the number of scorers strengthens the in-distribution performance, while LOBO performance is non-monotonic and does not improve in the same way. One plausible explanation is benchmark-specific overfitting. Averaging across multiple models reduces scorer-specific noise, but it may also reinforce shared sensitivity to benchmark-specific phrasing and formatting, limiting out-of-distribution transfer.

5.3 Contamination and feasibility

One diagnostic use of *ex ante* difficulty prediction is to compare predicted and observed difficulty after rollouts. Let $z(\beta_t)$ be the within-benchmark standardized difficulty implied by rollouts, and let $\hat{z}_t$ be the predicted task difficulty from the full ridge model in Section 5.1. Their difference

$$r_t = z(\beta_t) - \hat{z}_t \tag{5}$$

is the unexplained difficulty. If $r_t > 0$, agents found the task harder than the model predicted. If $r_t < 0$, agents found the task easier than the model predicted. The second case is the direction expected under familiarity or contamination, where agents succeed more often than the predicted difficulty would suggest. The first case is the direction expected when some missing factor, such as feasibility, makes the rollout harder than the task description suggests.

To illustrate this idea, we perform a small behavioral case study. An OpenAI audit reports that 59.4% of 138 frequently failed SWE-bench Verified tasks had material test-design or problem-description issues, and it also reports evidence of contamination for several tasks. Two of these audited examples appear in our dataset. For *django__django-14725*, flagged for missing prompt information, the residual equation yields $r_t = 1.46 - 0.05 = +1.41$. The task has success rate 0.090 and IRT estimated difficulty of $\beta = 2.58$, above the 90th percentile within SWE-bench Verified. For *django__django-11451*, discussed as a contamination example, the residual equation gives $r_t = -0.51 - 0.05 = -0.56$. The task has success rate 0.718 and IRT-estimated difficulty $\beta = -2.35$, below the benchmark median. Both residual signs match the intended interpretation. Unexpectedly hard tasks point toward feasibility or evaluator mismatch, while unexpectedly easy tasks point toward familiarity or contamination.

Because this residual uses empirical difficulty β_t from full agent rollouts, it supports detection rather than pure pre-rollout prediction. It remains an open question how far this mechanism can be pushed to automatically audit benchmark integrity and whether such anomalies can eventually be predicted without requiring execution at all.

6 Conclusion

We investigate whether the difficulty of agentic tasks can be estimated prior to execution. Our results show both promise and clear limits. Task features, particularly the sequential structure of token-level entropy, capture a meaningful difficulty signal, yet the results still reveal a clear divide between in- and out-of-distribution performance. While in-distribution rank recovery proves reliable, transferring these predictions to unseen benchmarks remains an open problem. We also show that AUC is unreliable for this setting. Because it conflates agent ability with task difficulty, it can remain artificially high even when estimates are degenerate, underscoring the need for rank-based evaluation.

Overall, we believe this work provides a foundation for more proactive environment design. If refined, *ex ante* prediction could serve as a diagnostic filter by using the residual gap between expected and observed difficulty to audit tasks for anomalies, including data contamination or structural infeasibility, before committing to exhaustive rollouts. Future work should focus on improving cross-benchmark generalization, for example by modeling entropy as a time series or by developing more mechanistic predictors of task familiarity and feasibility. As the cost of simulating frontier agents grows, such predictive diagnostics will become increasingly important for both training and evaluation.

References

Sanghwan Bae, Jiwoo Hong, Min Young Lee, Hanbyul Kim, JeongYeon Nam, and Donghyun Kwak. Online difficulty filtering for reasoning oriented reinforcement learning. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 700–719, 2026.

Longju Bai, Zhemin Huang, Xingyao Wang, Jiao Sun, Rada Mihalcea, Erik Brynjolfsson, Alex Pentland, and Jiaxin Pei. How do ai agents spend your money? analyzing and predicting token consumption in agentic coding tasks. *arXiv preprint arXiv:2604.22750*, 2026.

Mislav Balunovic, Jasper Dekoninck, Ivo Petrov, Nikola Jovanović, and Martin Vechev. Matharena: Evaluating llms on uncontaminated math competitions. *Advances in Neural Information Processing Systems*, 38, 2026.

Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th annual international conference on machine learning*, pp. 41–48, 2009.

Michael Beukman, Samuel Coward, Michael Matthews, Mattie Fellows, Minqi Jiang, Michael Dennis, and Jakob Foerster. Refining minimax regret for unsupervised environment design. *arXiv preprint arXiv:2402.12284*, 2024.

Mar Gonzàlez I Català, Haitz Sáez de Ocáriz Borde, George D Montañez, and Pietro Liò. The stepwise informativeness assumption: Why are entropy dynamics and reasoning correlated in llms? *arXiv preprint arXiv:2604.06192*, 2026.

Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, et al. Mle-bench: Evaluating machine learning agents on machine learning engineering. In *International Conference on Learning Representations*, volume 2025, pp. 50466–50494, 2025.

Thomas M Cover. *Elements of information theory*. John Wiley & Sons, 1999.

Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web: Towards a generalist agent for the web. *Advances in Neural Information Processing Systems*, 36:28091–28114, 2023.

Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, et al. Swe-bench pro: Can ai agents solve long-horizon software engineering tasks? *arXiv preprint arXiv:2509.16941*, 2025.

Michael Dennis, Natasha Jaques, Eugene Vinitsky, Alexandre Bayen, Stuart Russell, Andrew Critch, and Sergey Levine. Emergent complexity and zero-shot transfer via unsupervised environment design. *Advances in neural information processing systems*, 33:13049–13061, 2020.

Yihong Dong, Xue Jiang, Huanyu Liu, Zhi Jin, Bin Gu, Mengfei Yang, and Ge Li. Generalization or memorization: Data contamination and trustworthy evaluation for large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 12039–12050, 2024.

Susan E Embretson and Steven P Reise. *Item response theory: Foundations for psychologists and social scientists*. Routledge, 2025.

Xidong Feng, Vivek Veeriah, Marcus Chiam, Michael Dennis, Federico Barbero, Johan Obando Ceron, Jiaxin Shi, Satinder Singh, Shaobo Hou, Nenad Tomasev, et al. Generating creative chess puzzles. *Advances in Neural Information Processing Systems*, 38:49539–49605, 2026.

Carlos Florensa, David Held, Xinyang Geng, and Pieter Abbeel. Automatic goal generation for reinforcement learning agents. In *International conference on machine learning*, pp. 1515–1528. PMLR, 2018.

Chris Ge, Daria Kryvosheieva, Daniel Fried, Uzay Girit, and Kaivalya Hariharan. Agent psychometrics: Task-level performance prediction in agentic coding benchmarks. *arXiv preprint arXiv:2604.00594*, 2026.

Hila Gonen, Srini Iyer, Terra Blevins, Noah A Smith, and Luke Zettlemoyer. Demystifying prompts in language models via perplexity estimation. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 10136–10148, 2023.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

Alex Graves, Marc G Bellemare, Jacob Menick, Remi Munos, and Koray Kavukcuoglu. Automated curriculum learning for neural networks. In *international conference on machine learning*, pp. 1311–1320. Pmlr, 2017.

Lukas Helff, Quentin Delfosse, David Steinmann, Ruben Härle, Hikaru Shindo, Patrick Schramowski, Wolfgang Stammer, Kristian Kersting, and Felix Friedrich. Llms gaming verifiers: Rlvr can lead to reward hacking. *arXiv preprint arXiv:2604.15149*, 2026.

Anson Ho, Jean-Stanislas Denain, David Atanasov, Samuel Albanie, and Rohin Shah. A rosetta stone for ai benchmarks. *arXiv preprint arXiv:2512.00193*, 2025.

Matthew D Hoffman, David M Blei, Chong Wang, and John Paisley. Stochastic variational inference. *Journal of machine learning research*, 2013.

Naman Jain, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. In *International Conference on Learning Representations*, volume 2025, pp. 58791–58831, 2025.

Minqi Jiang, Edward Grefenstette, and Tim Rocktäschel. Prioritized level replay. In *International Conference on Machine Learning*, pp. 4940–4950. PMLR, 2021.

Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024.

Diederik P Kingma and Max Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.

Yehuda Koren, Robert Bell, and Chris Volinsky. Matrix factorization techniques for recommender systems. *Computer*, 42(8):30–37, 2009.

Stefan Krsteski, Charlotte Meyer, Guillaume Allegre, Tony O'Halloran, and Alexandre Sallinen. Messier: A high-resolution corpus for cross-benchmark agent evaluation. *arXiv preprint arXiv:2607.25891*, 2026.

M Kumar, Benjamin Packer, and Daphne Koller. Self-paced learning for latent variable models. *Advances in neural information processing systems*, 23, 2010.

Thomas Kwa, Ben West, Joel Becker, Amy Deng, Katharyn Garcia, Max Hasin, Sami Jawhar, Megan Kinniment, Nate Rush, Sydney Von Arx, et al. Measuring ai ability to complete long software tasks. *Advances in Neural Information Processing Systems*, 38:92213–92266, 2026.

Yucheng Li. Estimating contamination via perplexity: quantifying memorisation in language model evaluation. *URL https://arxiv. org/abs/2309.10677*, 2023.

Fengyuan Liu, Jay Gala, Dzmitry Bahdanau, Siva Reddy, Hugo Larochelle, et al. Bridge: Predicting human task completion time from model performance. *arXiv preprint arXiv:2602.07267*, 2026.

Ilia Mahrooghi, Aryo Lotfi, and Emmanuel Abbe. Goldilocks rl: Tuning task difficulty to escape sparse rewards for reasoning. *arXiv preprint arXiv:2602.14868*, 2026.

Tambet Matiisen, Avital Oliver, Taco Cohen, and John Schulman. Teacher–student curriculum learning. *IEEE transactions on neural networks and learning systems*, 31(9):3732–3740, 2019.

Nina Mazar, On Amir, and Dan Ariely. The dishonesty of honest people: A theory of self-concept maintenance. *Journal of marketing research*, 45(6):633–644, 2008.

Mike A Merrill, Alexander G Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E Kelly Buchanan, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *arXiv preprint arXiv:2601.11868*, 2026.

Team Olmo, Allyson Ettinger, Amanda Bertsch, Bailey Kuehl, David Graham, David Heineman, Dirk Groeneveld, Faeze Brahman, Finbarr Timbers, Hamish Ivison, et al. Olmo 3. *arXiv preprint arXiv:2512.13961*, 2025.

Sophie Ostmeier, Brian Axelrod, Maya Varma, Asad Aali, Yabin Zhang, Magdalini Paschali, Sanmi Koyejo, Curtis Langlotz, and Akshay Chaudhari. Attention head entropy of llms predicts answer correctness. *arXiv preprint arXiv:2602.13699*, 2026.

Jack Parker-Holder, Minqi Jiang, Michael Dennis, Mikayel Samvelyan, Jakob Foerster, Edward Grefenstette, and Tim Rocktäschel. Evolving curricula with regret-based environment design. In *International Conference on Machine Learning*, pp. 17473–17498. PMLR, 2022.

Shishir G Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E Gonzalez. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In *Forty-second International Conference on Machine Learning*, 2025.

Tejal Patwardhan, Rachel Dias, Elizabeth Proehl, Grace Kim, Michele Wang, Olivia Watkins, Simón Posada Fishman, Marwan Aljubeh, Phoebe Thacker, Laurance Fauconnet, et al. Gdpval: Evaluating ai model performance on real-world economically valuable tasks. *arXiv preprint arXiv:2510.04374*, 2025.

Long Phan, Alice Gatti, Ziwen Han, Nathaniel Li, Josephina Hu, Hugh Zhang, Chen Bo Calvin Zhang, Mohamed Shaaban, John Ling, Sean Shi, et al. Humanity's last exam. *arXiv preprint arXiv:2501.14249*, 2025.

Rémy Portelas, Cédric Colas, Lilian Weng, Katja Hofmann, and Pierre-Yves Oudeyer. Automatic curriculum learning for deep rl: A short survey. *arXiv preprint arXiv:2003.04664*, 2020.

Georg Rasch. *Probabilistic Models for Some Intelligence and Attainment Tests*. Copenhagen: Institute of Education Research, Copenhagen, 1960.

David Rein, Joel Becker, Amy Deng, Seraphina Nix, Chris Canal, Daniel O'Connel, Pip Arnott, Ryan Bloom, Thomas Broadley, Katharyn Garcia, et al. Hcast: Human-calibrated autonomy software tasks. *arXiv preprint arXiv:2503.17354*, 2025.

Claude Elwood Shannon. A mathematical theory of communication. *The Bell system technical journal*, 27(3):379–423, 1948.

Manish Shetty, Naman Jain, Jinjian Liu, Vijay Kethanaboyina, Koushik Sen, and Ion Stoica. Gso: Challenging software optimization tasks for evaluating swe-agents. *Advances in Neural Information Processing Systems*, 38, 2026.

Joar Skalse, Nikolaus Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward gaming. *Advances in Neural Information Processing Systems*, 35: 9460–9471, 2022.

Ari Spiesberger, Juan J Vazquez, Nicky Pochinkov, Tomáš Gavenčiak, Peli Grietzer, Gavin Leech, and Nandi Schoots. Soft contamination means benchmarks test shallow generalization. *arXiv preprint arXiv:2602.12413*, 2026.

Sidney Tio, Jimmy Ho, and Pradeep Varakantham. Training reinforcement learning agents and humans with difficulty-conditioned generators. *arXiv preprint arXiv:2312.02309*, 2023.

Sang Truong, Yuheng Tu, Percy Liang, Bo Li, and Sanmi Koyejo. Reliable and efficient amortized model-based evaluation. *arXiv preprint arXiv:2503.13335*, 2025.

L. S. Vygotsky. *Mind in Society: The Development of Higher Psychological Processes*. Harvard University Press, Cambridge, MA, 1978.

Hjalmar Wijk, Tao Lin, Joel Becker, Sami Jawhar, Neev Parikh, Thomas Broadley, Lawrence Chan, Michael Chen, Josh Clymer, Jai Dhyani, et al. Re-bench: Evaluating frontier ai r&d capabilities of language model agents against human experts. *arXiv preprint arXiv:2411.15114*, 2024.

Cheng Xu, Shuhao Guan, Derek Greene, M Kechadi, et al. Benchmark data contamination of large language models: A survey. *arXiv preprint arXiv:2406.04244*, 2024.

Frank Fangzheng Xu, Yufan Song, Boxuan Li, Yuxuan Tang, Kritanjali Jain, Mengxue Bao, Zora Wang, Xuhui Zhou, Zhitong Guo, Murong Cao, et al. Theagentcompany: benchmarking llm agents on consequential real world tasks. *Advances in Neural Information Processing Systems*, 38, 2026.

Tianci Xue, Weijian Qi, Tianneng Shi, Chan Hee Song, Boyu Gou, Dawn Song, Huan Sun, and Yu Su. An illusion of progress? assessing the current state of web agents. *arXiv preprint arXiv:2504.01382*, 2025.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Andy K Zhang, Neil Perry, Riya Dulepet, Joey Ji, Celeste Menders, Justin Lin, Eliot Jones, Gashon Hussein, Samantha Liu, Donovan Jasper, et al. Cybench: A framework for evaluating cybersecurity capabilities and risks of language models. In *International Conference on Learning Representations*, volume 2025, pp. 25094–25243, 2025.

Chenghua Zhu, Siyan Wu, Xiangkang Zeng, Zishan Xu, Zhaolu Kang, Yifu Guo, Yuquan Lu, Junduan Huang, and Guojing Zhou. Edis: Diagnosing llm reasoning via entropy dynamics. *arXiv preprint arXiv:2602.01288*, 2026.

A Supplementary material

A.1 Benchmark list

Benchmark	Primary domain
BFCL Live (Patil et al., 2025)	Function calling
BFCL Multi-Turn (Patil et al., 2025)	Multi-turn function calling
CyBench (Zhang et al., 2025)	Cybersecurity tasks
GDPval (Patwardhan et al., 2025)	Economically valuable real-world tasks
GSO (Shetty et al., 2026)	Software optimization
HCAST (Rein et al., 2025)	Human-calibrated autonomy tasks
HLE (Phan et al., 2025)	Broad knowledge and reasoning
LiveCodeBench (Jain et al., 2025)	Code generation
MathArena (Balunovic et al., 2026)	Competition mathematics
MLE-bench (Chan et al., 2025)	Machine-learning engineering
Online-Mind2Web (Xue et al., 2025; Deng et al., 2023)	Web navigation
RE-Bench (Wijk et al., 2024)	Research-engineering tasks
SWAA (Kwa et al., 2026)	Short-horizon autonomous-agent tasks
SWE-bench Pro (Deng et al., 2025)	Software engineering
SWE-bench Verified (Jimenez et al., 2024)	Software engineering
Terminal-Bench (Merrill et al., 2026)	Terminal tasks
TheAgentCompany (Xu et al., 2026)	Office-style multi-tool tasks

Table 3: Benchmarks and primary domains.

A.2 IRT fitting details

The IRT model is fit on the observed binary entries of the agent-task response matrix. We place zero-centered Gaussian priors with learned global scales on each agent ability θ_a and task difficulty β_t, approximate the posterior for each scalar with a normal variational factor, and optimize the evidence lower bound with stochastic variational inference. The posterior means are then used as the operational ability and difficulty estimates. The fitted scores $\theta_a - \beta_t$ achieve response AUC 0.937 on observed pairs, indicating that the fitted scale captures the response matrix well enough to define the difficulty target used in the paper.

Fitting rationale. As in matrix factorization (Koren et al., 2009), only observed response-matrix entries contribute to the fit. A direct joint maximum-likelihood (JML) fit would initialize all abilities and difficulties and minimize binary cross-entropy over observed trials. This point-estimate fit is fragile under imbalance. Parameters with few observations become noisy, and the loss sees only the error in $\theta_a - \beta_t$. If θ_a is poorly estimated, a mispredicted trial can still push the paired β_t in the opposite direction, transferring noise between ability and difficulty.

The natural correction is to avoid committing to a single latent value too early. In the standard ability-marginalization view, for fixed task difficulties, each agent contribution is obtained by averaging over plausible abilities. Let $\Omega_a = \{t : (a,t) \in \Omega\}$ be the observed tasks for agent a. Then

$$p(Y_{a,\Omega_a} \mid \boldsymbol{\beta}) = \int p(\theta_a) \prod_{t \in \Omega_a} p(Y_{a,t} \mid \theta_a, \beta_t)\, d\theta_a.$$

A grid approximation with K support points requires $O(K|\Omega|)$ likelihood evaluations per iteration, about 4.15×10^6 evaluations for $K = 10$ at our scale, before repeated iterations. The variational fit replaces this grid enumeration with learned Gaussian approximate posteriors. The posterior means provide point estimates, while the posterior scales keep an uncertainty estimate for each ability and difficulty.

Implementation details. The fit uses a 1PL variational IRT implementation with Trace_ELBO optimized by ClippedAdam for 2,000 epochs, learning rate 0.01, gradient

Estimator	N	Dim.	KF $\rho \uparrow$	KF pAcc $\uparrow$	LOBO $\rho \uparrow$	LOBO pAcc $\uparrow$
Lin.	5,230	217	0.451 ± 0.034	0.676 ± 0.012	$\mathbf{0.295 \pm 0.230}$	$\mathbf{0.602 \pm 0.085}$
Ridge	5,230	217	0.454 ± 0.034	0.678 ± 0.013	0.277 ± 0.259	0.597 ± 0.101
RF	5,230	217	$\mathbf{0.475 \pm 0.033}$	$\mathbf{0.683 \pm 0.015}$	0.130 ± 0.250	0.545 ± 0.089
XGB	5,230	217	0.453 ± 0.037	0.678 ± 0.013	0.190 ± 0.199	0.563 ± 0.071

Table 4: Estimator choice does not remove the held-out-benchmark gap. Lin.=linear regression, RF=random forest, XGB=XGBoost. KF uses 10 folds; LOBO leaves out one benchmark. pAcc denotes within-benchmark pairwise accuracy. Entries report mean $\pm$ standard deviation across folds or held-out benchmarks.

clipping at norm 5.0, and random seed 42. The saved state contains θ, β, and their posterior scales for 502 agents and 7,266 tasks before filtering to the main analysis subset.

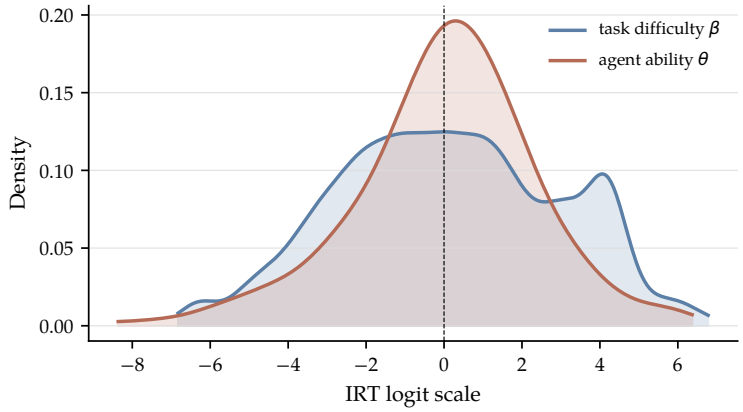


Figure 4: **Fitted IRT ability and difficulty distributions.** Kernel-density estimates are shown for item difficulty β and agent ability θ before filtering to the main analysis subset.

A.3 Entropy scorer panel

The single-scorer *entropy* setting uses Qwen3-8B-Base. The *full* setting uses the following five-scorer panel, in this fixed order for the scorer-count ablation: Qwen3-8B-Base; Qwen3-8B-Instruct, abbreviated as Qwen3-8B in Figure 3 (Yang et al., 2025); Gemma 4 E4B, abbreviated as Gemma-E4B-it in Figure 3; OLMo 3 7B Instruct, abbreviated as OLMo-3-7B-Inst (Olmo et al., 2025); and Llama 3.1 8B Instruct, abbreviated as Llama-3.1-8B-Inst (Grattafiori et al., 2024).

A.4 Estimator ablation

Figure 2 fixes the prediction protocol to compare feature families. Table 4 instead fixes the full feature representation and varies the regressor. Random forests improve KF performance slightly, reaching $\rho = 0.475$ and pairwise accuracy 0.683. Linear regression gives the strongest LOBO result, with $\rho = 0.295$ and pairwise accuracy 0.602. The ablation therefore leaves the main conclusion unchanged. Estimator choice affects interpolation within the pooled dataset, while benchmark transfer remains the limiting case.

A.5 Human-time calibration

Table 5 calibrates fitted IRT difficulty to log median human completion time on the subset with human-time annotations, following the BRIDGE framing (Liu et al., 2026). Linear and ridge regression perform nearly identically, with KF $\rho = 0.422$ and MAE 0.857. XGBoost gives the lowest KF MAE, 0.849, but weaker LOBO rank correlation. These results are

Estimator	N	KF $\rho \uparrow$	KF MAE $\downarrow$	LOBO $\rho \uparrow$	LOBO MAE $\downarrow$
Lin.	1,676	**0.422 ± 0.060**	0.857 ± 0.016	**0.464 ± 0.277**	**0.863**
Ridge	1,676	**0.422 ± 0.060**	0.857 ± 0.015	**0.464 ± 0.277**	**0.863**
RF	1,676	0.304 ± 0.055	0.922 ± 0.026	0.292 ± 0.182	0.957
XGB	1,676	0.408 ± 0.056	**0.849 ± 0.015**	0.391 ± 0.178	0.886

Table 5: Human-time calibration of fitted IRT difficulty, following the BRIDGE framing (Liu et al., 2026). Lin.=linear regression, RF=random forest, XGB=XGBoost. KF uses 5 folds. LOBO leaves out one benchmark. Uncertainties are standard deviations across folds. MAE is measured on the log median-human-minutes target.

auxiliary to the main paper, but they indicate that the fitted difficulty scale can be connected to human-interpretable task time when such annotations are available. Practically, accurate pre-rollout difficulty prediction could help researchers estimate human-time requirements more efficiently.

A.6 OLMo surprisal

To sanity-check the use of language-model likelihood as a lightweight surprise signal, we scored a small set of prompt categories with `allenai/OLMo-1B-hf`. The experiment uses only short text snippets and should be read as a qualitative diagnostic rather than a benchmark result. For each prompt, we compute mean token surprisal, $-\log p$, in nats per token under the model. Higher bars, therefore, indicate text that is more surprising to the scorer model.

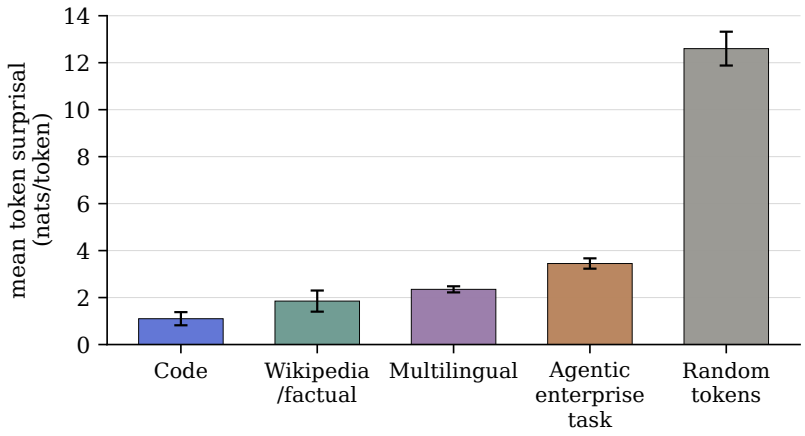


Figure 5: Mean token surprisal under OLMo-1B across prompt categories, measured in nats per token. Bars show means over five prompts per category; error bars show standard deviations. Random-token strings are included as a high-surprise control.